ERDOS RESEARCH
Position Paper · Forward Deployed Engineering

Expanding the Role of the Forward Deployed Engineer

Beyond a Customer-Facing Engineer

The Forward Deployed Engineer is usually defined as a customer-facing engineer who embeds with a client to ship software. This paper argues for a broader definition: the FDE is the function that reduces uncertainty and converts tacit organisational judgement into executable, evaluable AI systems.

By **Ajit Jaokar**
Founder & Lead Researcher, Erdos Research
Course Director, Artificial Intelligence, University of Oxford

Contents

Introduction

Today, the Forward Deployed Engineer (FDE) is defined largely in technical terms — a customer-facing engineer who embeds with a client to ship working software. That definition is accurate, but it is narrow, and there is considerable value in expanding it.

At its broadest, **an FDE transforms organisational knowledge into executable AI systems.** A more precise way to put it: an FDE reduces the distance between a customer problem and a working solution.

Where a software engineer's primary asset is code, and a consultant's is analysis, the FDE combines business understanding, systems thinking, architecture, product judgement, and implementation into a single role. This leads to the central claim:

> **The central claim**
>
> **FDEs are not primarily building software. They are reducing uncertainty.**

The claim sounds strange at first, because FDEs plainly do write code, build systems, deploy agents, and design architectures. But those activities are not the goal — they are the mechanisms. The goal is to move a project from assumption to evidence. The role breaks down into ten capabilities.

1. Domain Knowledge

The FDE understands the problem before designing the solution, developing a deep grasp of the industry, the business, its users, workflows, decisions, regulations, and constraints. Unlike a traditional engineer, the FDE is expected to understand how the business thinks, not just how software works.

Primary outcome: *Capture expert knowledge.*

2. Translation of Requirements

This is arguably the defining capability of the role. The FDE translates across multiple layers of abstraction — Business → Processes → Requirements → Architecture → Agents → Code — converting business goals, stakeholder expectations, tacit knowledge, policies, workflows, and decision logic into PRDs, agent specifications, prompts, tool definitions, evaluation criteria, and governance rules.

Primary outcome: *Convert ambiguity into executable specifications.*

3. AI-Assisted Development

Modern FDEs no longer write every line of software; they orchestrate AI-assisted engineering using tools such as Claude Code, OpenAI Codex, GitHub Copilot, Cursor, Lovable, and Windsurf. The FDE becomes the conductor rather than the typist, shifting toward decomposing problems, supervising AI output, validating results, iterating quickly, and maintaining architectural integrity.

Primary outcome: *Maximise engineering productivity with AI.*

4. Reducing Uncertainty

This is the core responsibility. Every enterprise AI project begins in a state of uncertainty: Is this the right problem? Can AI actually solve it? Which model, architecture, workflow, or data should be used? What are the risks, and how is success measured? The FDE reduces this uncertainty through a disciplined evidence sequence — Discovery → Prototype → Evaluation → Governance → Production → Continuous Improvement.

Primary outcome: *Replace assumptions with evidence.*

5. Knowledge Engineering

This is the capability most current definitions overlook. The FDE captures organisational intelligence — tacit expertise, decision-making patterns, institutional memory, workflows, policies, best practices, and knowledge graphs — and converts it into prompts, memory, RAG systems, planning logic, evaluation frameworks, and governance structures.

Primary outcome: *Transform human expertise into executable knowledge.*

6. Enterprise AI Governance

Production AI is impossible without governance. The FDE operationalises it through guardrails, human approval steps, audit trails, LLM-as-a-Judge evaluation, monitoring, compliance checks, and safety controls.

Primary outcome: *Make AI trustworthy.*

7. Systems Thinking

The FDE sees the entire enterprise system. Rather than optimising a single workflow in isolation, they optimise the interactions between people, data, software, agents, and processes.

Primary outcome: *Optimise the system, not the component.*

8. Agent Architecture

The FDE designs multi-agent systems — orchestration, planning, memory, tools, and workflows — thinking in terms of autonomous capabilities rather than isolated prompts.

Primary outcome: *Design capabilities, not prompts.*

9. Business Value Realisation

AI is not the product; business capability is. The FDE ultimately measures adoption, ROI, cost reduction, quality improvements, customer satisfaction, and organisational capability.

Primary outcome: *Deliver capability, not technology.*

10. Implementation and Validation

The final question is the simplest: does it actually work? The FDE proves the system performs against real workflows and real data — in production, not in a demo.

Primary outcome: *Prove it works in production, not in a pilot.*

Taken together, these ten capabilities describe a single motion. The FDE converts what is human, tacit, and ambiguous — knowledge, goals, expertise, judgement — into what is executable,

evaluable, and reliable. Every artefact they produce, from a prompt to a governance rule, is an instrument for turning assumption into evidence. That, rather than code, is the real output of the role.

The Forward-Deployed Engineer's Real Output Is “Decision Artefacts”

The act of reducing uncertainty leads to the creation of decision artefacts. Software engineers build systems. Forward-Deployed Engineers build solutions.

A traditional software engineer's deliverable is, in the end, software. An FDE's deliverable is a working business capability — and the most durable part of that capability is rarely the code. It is the trail of decisions made explicit along the way (decision artefacts).

Every time an FDE resolves an ambiguous problem into something a system can actually do, a decision gets made. The artefact translates individual decisions into something an organisation can review, challenge, and reuse. This role becomes even more significant in the age of Agentic AI and AI-assisted development.

What a decision artefact actually is

A decision artefact is not a document for its own sake. It is the trace of a choice, captured with enough context that someone who wasn't in the room can reconstruct the reasoning. A good one records:

- **The decision** — what was chosen.
- **The problem it answers** — the question that forced the choice.
- **The alternatives considered** — and why they were rejected, not just which one won.
- **The assumptions** — the conditions under which the decision holds.
- **The trade-offs** — what was given up to gain something else.
- **The trigger for revisiting** — what would have to change for this decision to be wrong.

That last property is the one people forget, and it is the one that matters most in AI. Decisions made under uncertainty have a shelf life. An artefact that names its own expiry conditions is far more valuable than one that pretends to be permanent. Decision artefacts can be implemented through context graphs.

> **The four things a decision artefact must be**
>
> **Reviewable** — someone can inspect the reasoning. **Challengeable** — someone can disagree with it on the merits.
> **Reusable** — the next use case can build on it. **Auditable** — a regulator or risk committee can trace it later.

Software alone gives you none of these. It tells you what the system does, never why it was allowed to.

Why AI makes this more important, not less

You could argue every engineering project produces decisions worth recording. But three features of AI systems push decision artefacts from “good practice” to “the actual job.”

AI is built under deep uncertainty. You cannot fully specify an agent's behaviour up front the way you specify a deterministic function. You discover what the model can and can't do by building against it. That means requirements shift mid-flight, and the decisions that absorb those shifts — scope, behaviour, guardrails — carry unusual weight. Capturing them is how you keep the project coherent while the ground moves.

Behaviour is probabilistic, so intent has to be written down. When a system's output is a distribution rather than a fixed value, "the code is the spec" stops being true. The specification lives in the decisions about how the agent should behave, what it must never do, and what counts as acceptable. If those aren't artefacts, they're folklore.

Enterprise AI is governed. Auditors, regulators, and internal risk functions will ask why — why this model, why these controls, why you judged it safe to deploy. An answer that exists only in the engineers' heads is not an answer. Governance is, in large part, the discipline of having your decision artefacts ready before anyone asks for them.

So the shift is this: in traditional software the code is the deliverable and the decisions are supporting evidence. In enterprise AI the decisions are the deliverable and the code is one of their consequences.

The core mapping: every question is an artefact

The cleanest way to see an FDE's work is to list the questions that must be answered before a system can responsibly ship — and notice that each one resolves into an artefact.

The decision to be made	The artefact it produces
Should we build this at all?	Business case
What exactly are we solving?	Problem statement
How should the agent behave?	Agent specification
Which model should we use?	Model selection rationale
What tools can it call?	Tool specification
How will we know it works?	Evaluation plan
How will we know it's safe?	Governance and risk assessment
How will we measure value?	Business KPI framework
How will it evolve?	Continuous improvement roadmap

These aren't engineering aspects; they're the choices an organisation is accountable for. The FDE's contribution is to force each question into the open and leave behind an artefact that survives the person who made it.

Decisions cluster along the lifecycle

Of course, decision artefacts aren't produced all at once. They accumulate across the delivery lifecycle, and it helps to see which decisions live at each stage.

Discover — deciding what's worth solving. The artefacts here settle scope and justify investment: problem definition, opportunity assessment, stakeholder analysis, current-workflow mapping, an AI-suitability assessment, prioritised use cases, and the success metrics that will later judge everything else.

Governing decision: *Is this the right problem, and is AI the right tool for it?*

Define — deciding what "good" means. AI requirements differ from functional specs because uncertainty has to be captured explicitly rather than assumed away. The artefacts include the PRD, agent and persona specifications, tool and knowledge requirements, memory and context requirements, human-in-the-loop requirements, guardrails, and evaluation criteria.

Governing decision: *What must be true for this to count as working?*

Design — deciding how it will be built. Here the FDE bridges business and engineering: end-to-end and agent architecture, workflow and data-flow diagrams, retrieval and tool-integration design, API specifications, and the security and governance architecture. Alongside these sit the design decisions unique to modern AI — the prompt and skill libraries, harness definitions, planning strategy, multi-agent interaction patterns, and the memory, context, and error-handling strategies.

Governing decision: *What structure will make this reliable?*

Govern — deciding what makes it safe. This has become one of the defining responsibilities of the enterprise FDE. The artefacts are the risk assessment, governance checklist, policy and regulatory mapping, human approval points, escalation workflows, audit-trail and logging design, compliance evidence, and responsible-AI controls.

Governing decision: *Under what conditions are we willing to let this act?*

Build & Evaluate — deciding whether it's real. Building produces the backlog, technical specifications, integration and test plans, and rollout strategy. Evaluation produces the evidence: technical measures (accuracy, latency, cost, reliability), AI-specific measures (hallucination, groundedness, retrieval, tool-use, planning, robustness, and safety evaluation), and business measures (time saved, revenue impact, adoption, ROI).

Governing decision: *Does the evidence justify the claim that this works?*

Deploy & Improve — deciding how it endures. After launch, the FDE keeps deciding: usage and evaluation dashboards, failure analysis, incident reports, cost optimisation, prompt and workflow improvements, model-upgrade recommendations, and periodic governance reviews.

Governing decision: *What have we learned, and what does it change?*

Notice that "the software" appears in exactly one of those phases. Everything around it is judgement, made legible.

The consolidated view

The FDE's complete output for an enterprise AI system falls into six kinds of artefact:

1. **Business understanding** — the problems, workflows, stakeholders, and value at stake.
2. **Structured judgement** — the documented decisions, trade-offs, assumptions, and rationale. *This is the decision-artefact layer in its purest form.*
3. **AI system design** — agents, workflows, prompts, tools, memory, and integrations.
4. **Enterprise governance** — risk controls, compliance, auditability, and human oversight.
5. **Evidence** — evaluations demonstrating technical performance, safety, and business impact.
6. **Operational readiness** — deployment, monitoring, incident response, and continuous improvement.

Every one of these is a container for decisions. Business understanding records what we decided to care about. Governance records what we decided to allow. Evidence records what we decided counts as proof. Even the system design is a stack of resolved choices about behaviour and structure.

Implications of decision artefacts as an outcome

If we treat each major decision as an artefact — something to be reviewed, challenged, and reused — two things happen. The project becomes legible to the people who have to stand behind it. And the organisation gets better at the next one, because judgement that used to evaporate now compounds.

That is the real shift in AI engineering: from delivering code to delivering trusted, measurable, continuously improving capability. The code is where the work becomes visible. The decision artefacts are where it becomes durable.

Who Consumes the Deliverables of the FDE?

The above discussion leads us to the next logical question: who consumes the deliverables of the FDE?

Every FDE artefact is addressed to someone with a decision to make

If a Forward-Deployed Engineer's real output is a trail of decisions made explicit, then the obvious next question is: explicit to whom? A decision artefact that no one reads, challenges, or acts on isn't an artefact. It's paperwork. This is where the FDE role diverges most sharply from traditional software engineering.

A software engineer usually has one or two consumers — another engineer, a product manager — and a single register to write in. An FDE writes for an entire organisation. The business case goes to a board. The guardrail mapping goes to a governance committee. The tool permissions go to security. The same underlying system produces a dozen different audiences, each holding a different decision.

So it helps to stop thinking about deliverables and start thinking about consumers — because the consumer is the artefact's real specification. Who has to act on this decision determines its language, its altitude, and its level of detail. The FDE's job is not just to make decisions explicit, but to route each one to the person accountable for it, in a form they can use.

The consumer of the FDE output drives the spec

Consider the same decision — "how should this agent behave?" — as it travels. Executive leadership need it as a one-line risk posture. A business stakeholder needs it as a change to their workflow. A product manager needs it as acceptance criteria. An AI engineer needs it as an agent specification with guardrails and tool definitions. A governance team needs it as an approval point with an audit trail. One decision, five faces. The FDE writes all five.

That is why "who consumes this?" is not an administrative question. It is the question that tells you whether an artefact is finished. An agent specification that an engineer can build from but a risk committee can't reason about is only half-done. Here is the whole audience, compressed into the shape that matters — the decision each one actually owns.

Consumer	The decision they own	What they consume from the FDE
Executive leadership	Should we invest, and what risk are we accepting?	Business case, ROI analysis, prioritised use cases, risk summary
Business stakeholders	Does this solve our problem, and will people use it?	Workflow maps, user journeys, process redesign, agent capabilities
Product managers	What gets built, and in what order?	PRD, user stories, scope, acceptance criteria, roadmap
AI engineers	How do we build the intelligence?	Agent specs, prompt library, tool definitions, model selection, eval plan

Consumer	The decision they own	What they consume from the FDE
Software engineers	How do we integrate and ship it?	APIs, architecture, data contracts, auth flows, deployment requirements
Data teams	What data feeds it, and how?	Data and retrieval requirements, knowledge architecture, eval datasets
Security teams	Can we let this act safely?	Threat models, security architecture, tool permissions, attack surface
Governance teams	Under what controls is this allowed?	Guardrails, approval points, audit design, responsible-AI documentation
Legal & compliance	Are we within the law and our contracts?	Regulatory mapping, privacy impact, AI Act evidence, data handling
Operations / SRE / IT	Can we run and support it?	Monitoring plan, incident response, cost and capacity, runbooks
QA / evaluation	Does it actually work, and stay working?	Test cases, benchmarks, thresholds, regression suites, judge criteria
End users	(they decide nothing here)	A better system — which is the point of all the above

Read down the middle column and you have the anatomy of how an organisation actually adopts AI: a sequence of decisions, each owned by someone different, each needing evidence in its own language.

Six families of decision

The twelve consumers aren't twelve unrelated audiences. They cluster into a natural arc — authorise, scope, build, constrain, operate, experience — and it is worth walking it, because the arc is the delivery.

Those who decide whether it happens. Executive leadership and business stakeholders sit at the front. Leadership are answering four questions — should we invest, is this aligned to strategy, what value will we create, what risk are we accepting — and they consume the business case, ROI analysis, prioritised use cases, and risk summary as inputs to that call. Business stakeholders across operations, HR, finance, marketing, legal, and customer service are answering a narrower but harder one: does this actually solve our problem, and will people use it? They validate against workflow maps, user journeys, process redesign, and the agent's stated capabilities. If these two groups don't buy the decisions, nothing downstream matters.

Those who decide what gets built. Product managers turn intent into an executable plan. They consume the PRD, user stories, scope, acceptance criteria, and roadmap, and convert them into the implementation backlog. This is a translation decision: what, precisely, are we committing to build first?

Those who build it. Three groups share the construction decisions. AI engineers consume almost everything technical — agent specifications, the prompt and skill libraries, tool

definitions, knowledge sources, context-engineering strategy, model selection, workflow diagrams, and the evaluation plan. The FDE's contribution here is to reduce ambiguity before engineering begins, so the hard interpretive work is already done. Software engineers consume the integration surface: APIs, architecture, data contracts, database schema, authentication flows, and deployment requirements. Data teams consume what feeds the intelligence: data and retrieval requirements, knowledge architecture, embedding strategy, data-quality requirements, and evaluation datasets.

Those who decide what it's allowed to do. This family has grown from an afterthought into a defining audience for enterprise FDEs. Security teams decide whether the system can be permitted to act, using threat models, the security architecture, the identity model, tool permissions, secrets management, and attack-surface analysis. Governance teams decide under what controls it operates, consuming policy mapping, guardrails, human approval points, audit design, evaluation evidence, and responsible-AI documentation. Legal and compliance decide whether the whole thing sits inside the law and the contracts, reading regulatory mapping, data-handling documentation, privacy impact assessments, copyright considerations, AI Act evidence, and contractual assumptions. Their shared decision: not “does it work?” but “are we allowed to let it?”

Those who keep it alive and prove it works. Operations, SRE, and IT decide whether the system can be run and supported at all — they consume the monitoring plan, deployment strategy, incident-response procedures, cost estimates, capacity planning, and operational runbooks. QA and AI-evaluation teams decide whether it works and keeps working, consuming test cases, evaluation datasets, benchmark definitions, success thresholds, regression suites, LLM-as-a-Judge criteria, and human-review procedures. These are the decisions that stop a launch from quietly degrading into an incident.

The FDE as router of judgement

The FDE is not primarily a builder of systems. They are a router of judgement — the person who takes a consequential decision and delivers it to whoever is accountable for it, framed so they can actually act.

That reframes the measure of the role. The question isn't only “did the system get built?” It is “did the right decision reach the right desk, in a form that desk could use?” Did leadership get a risk summary they could weigh, not an engineering document they couldn't parse? Did security get permissions they could reason about, not a promise that it's fine? Did the governance team get evidence, not assurances?

An artefact with no consumer is waste. An artefact addressed to the wrong consumer is worse — it looks like diligence while helping no one. The enterprise value of an FDE lies in getting this right at scale: a dozen audiences, a dozen decisions, each served in its own language, all pointing at the same trustworthy system the end user never has to think about.

That is the quiet mechanics of adopting AI safely. Not the code, and not even the decisions alone — but the decisions delivered to the people who have to stand behind them.

Why Domain Knowledge Matters to the Role of the FDE

The FDE sits at the centre of the enterprise, translating between different perspectives. Every deliverable produced by the FDE is consumed by one or more of these groups.

Large language models already know a great deal about programming. What they don't know is:

- how a hospital actually works
- why traders ignore certain market signals
- why SOC analysts distrust specific alerts
- why manufacturing engineers override automation
- why lawyers interpret contracts differently
- why pilots follow checklists in a particular order

Those are examples of **institutional and tacit knowledge**. The FDE's job is to surface that knowledge and encode it into an AI system.

The FDE as a translator

It is useful to describe the FDE as translating between four worlds: domain experts, business, engineering, and AI systems. Every conversation is translated into something another group can use. For example, a clinician says: “This patient should probably be escalated.” The FDE asks:

- What does “probably” mean?
- What evidence supports escalation?
- What confidence threshold?
- What exceptions?
- Who approves?
- What happens afterwards?

The result becomes requirements, workflows, guardrails, and evaluation criteria.

Domain knowledge becomes AI knowledge

One of the FDE's core responsibilities is converting expert knowledge into structured representations.

Domain knowledge	AI representation
Clinical expertise	Decision workflow
Legal reasoning	Policy rules
Cyber analyst judgement	Detection and triage logic
Manufacturing procedures	Agent workflow
Customer service playbooks	Conversation design
Organisational policies	Guardrails

Domain knowledge	AI representation
Standard operating procedures	Skills and tools
Human expertise	Evaluation datasets

This transformation is one of the most valuable activities in enterprise AI.

Why this matters more in agentic AI

Traditional software automates **processes**. Agentic AI increasingly automates **decisions**. Decisions are rooted in domain expertise.

For example, a cybersecurity analyst doesn't simply classify alerts — they weigh business context, historical incidents, attacker behaviour, operational priorities, and organisational risk tolerance. Capturing that judgement is more important than implementing the API calls.

Similarly, in healthcare, the critical challenge is rarely calling an LLM. It is encoding the clinical reasoning, escalation pathways, safety constraints, and regulatory requirements that experienced clinicians use every day.

Conclusion

This article makes the claim that FDEs are not primarily building software; they are reducing uncertainty.

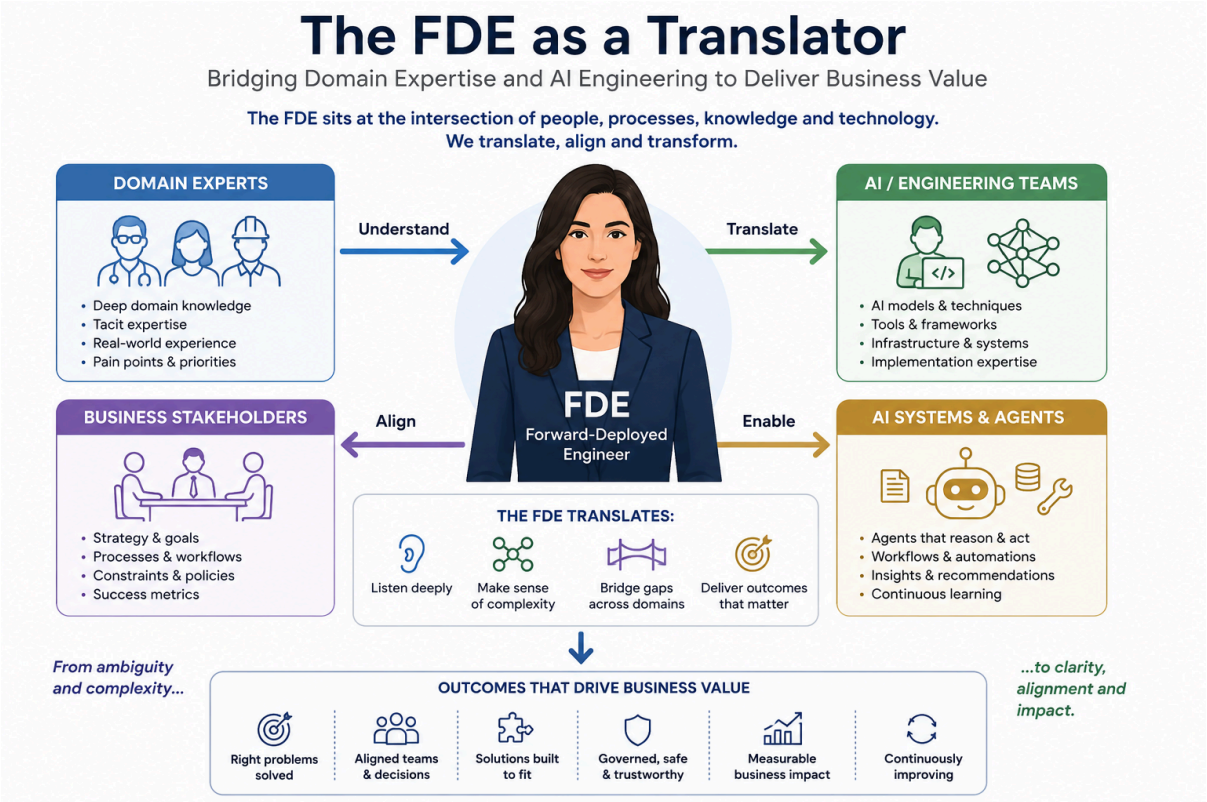
The ten capabilities describe the range of the role, and the FDE is the function that converts tacit organisational judgement into artefacts that are explicit, addressed, and evaluable — and does so under the specific condition that AI systems behave probabilistically rather than deterministically. This has a practical implication for how organisations and skills evolve.

It also means that we should not measure the FDE role by the software it ships. Instead, we should measure the success of the FDE by the decisions it surfaces, and by whether each decision is manifested into enterprise systems.

The deeper shift, then, isn't organisational — it's epistemic.

> Traditional engineering asks: *"does the system do what we specified?"*
>
> Enterprise AI, because it cannot be fully specified in advance, has to ask a prior question first: *"have we made explicit enough of what we actually mean, want, and will tolerate, that the answer to the first question is even checkable?"*

The FDE exists to answer that prior question, continuously, for every audience in the enterprise that has a stake in the answer. That is the real job of the FDE. The software is just the mechanism to make that role visible.


The FDE as a Translator
Bridging Domain Expertise and AI Engineering to Deliver Business Value
The FDE sits at the intersection of people, processes, knowledge and technology.
We translate, align and transform.
DOMAIN EXPERTS
Deep domain knowledge
Tacit expertise
Real-world experience
Pain points & priorities
Understand
Translate
AI / ENGINEERING TEAMS
AI models & techniques
Tools & frameworks
Infrastructure & systems
Implementation expertise
FDE
Forward-Deployed Engineer
BUSINESS STAKEHOLDERS
Strategy & goals
Processes & workflows
Constraints & policies
Success metrics
Align
Enable
AI SYSTEMS & AGENTS
Agents that reason & act
Workflows & automations
Insights & recommendations
Continuous learning
THE FDE TRANSLATES:
Listen deeply
Make sense of complexity
Bridge gaps across domains
Deliver outcomes that matter
From ambiguity and complexity...
OUTCOMES THAT DRIVE BUSINESS VALUE
Right problems solved
Aligned teams & decisions
Solutions built to fit
Governed, safe & trustworthy
Measurable business impact
Continuously improving
...to clarity, alignment and impact.